End-to-End CI/CD Pipeline for a Calculator REST API

CSE 816 Software Production Engineering · Mini-Project Report

Name: Digvijaysinh Pawar **Roll No.:** IMT2023099
Faculty: Prof. B. Thangaraju **Term:** I, 2026–27

GitHub: https://github.com/ImNotFound7/calculator-devops
Docker Hub: https://hub.docker.com/r/imnotfound/calculator-devops

1. Introduction

The goal of this project is to automate the delivery of a small Spring Boot calculator service. After a push to `main` on GitHub, Jenkins compiles the code, runs the tests, builds a Docker image tagged with the build number, pushes it to Docker Hub, redeploys the stack with Docker Compose, checks that it is healthy and emails me the result, with no manual steps. Stack: Java 17, Spring Boot 3.5, Maven, JUnit 5, Git/GitHub, Jenkins, Docker, Docker Compose and Nginx.

2. Architecture

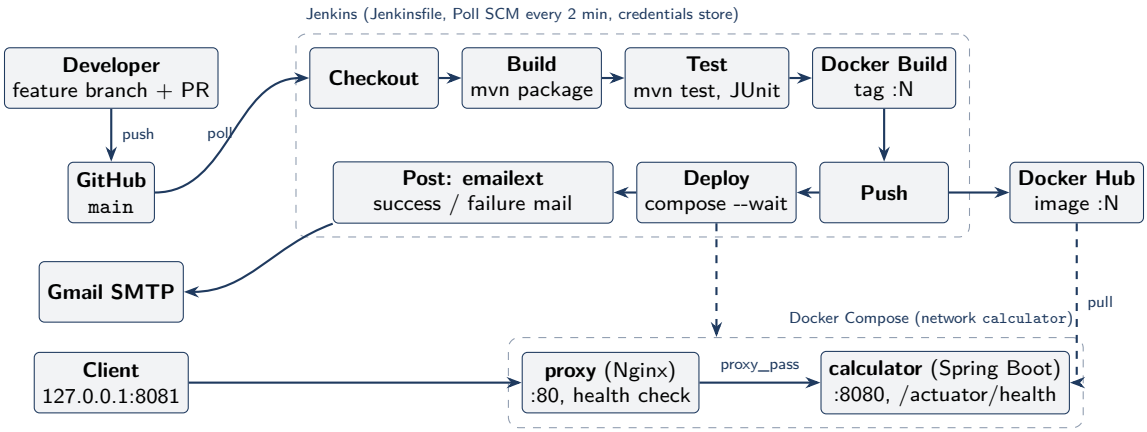


Figure 1: Pipeline and deployment architecture

Jenkins runs locally and deploys to the same Docker daemon. Only the Nginx proxy publishes a port, so clients reach the API through it.

3. Application and Tests

One controller handles `GET /api/v1/{add|subtract|multiply|divide}?a=&b=`. A request returns JSON such as `{"operation":"add","a":2.0,"b":3.0,"result":5.0}`, and Spring Actuator exposes `/actuator/health`. A `@RestControllerAdvice` returns HTTP 400 with a problem-detail body (`type`, `title`, `status`, `detail`, `instance`) for missing or non-numeric parameters, `NaN`/`Infinity`, division by zero (including −0.0) and overflow. Unknown operations return 404.

`mvn test` runs **31 JUnit 5 tests, all passing**: 14 unit tests in `CalculatorServiceTest` (every operation, divide-by-zero, non-finite input, overflow) and 17 MockMvc integration tests in `CalculatorApiTest` (JSON of each endpoint, 11 invalid requests, health, 404). Surefire writes XML reports that Jenkins publishes.

4. Version Control

The application and pipeline were developed on `feature/calculator-api` and merged through pull request #1; this report and the README cleanup came through a second pull request. Commits use `feat:`/`ci:`/`docs:` prefixes. `.gitignore` excludes `target/`, IDE files and local secrets (`.env`, `*.pem`, `*.key`). No credentials are committed.

5. Jenkins Pipeline

The declarative `Jenkinsfile` is loaded with *Pipeline script from SCM*. `pollSCM('H/2 * * * *')` triggers a build when `main` changes. I used Poll SCM rather than a webhook because Jenkins runs on my laptop, which GitHub cannot reach.

Stage	What it does
Checkout	`checkout scm` fetches the triggering commit.
Build	`mvn -B clean package -DskipTests` produces the executable JAR.
Test	`mvn -B test`; `junit` publishes reports in `post { always }`. A failure stops the pipeline.
Docker Build	`docker build -t imnotfound/calculator-devops:$BUILD_NUMBER .`
Push	`withCredentials` reads the `dockerhub` token; login via `--password-stdin` into a temporary `DOCKER_CONFIG` that is deleted afterwards.
Deploy	`docker compose up -d --no-build --wait`, then `/actuator/health` is requested through Nginx.
Post	`emailext` sends the status, build URL and image tag via Gmail SMTP; on failure, Compose logs are printed.

Docker Hub and Gmail secrets are *Username with password* entries in the Jenkins credentials store (`dockerhub`, `calculator-smtp`). Re-runs are safe: `disableConcurrentBuilds()` serialises deployments, Compose recreates only changed containers, and `--wait` completes only once the health checks pass.

6. Docker and Docker Compose

Dockerfile. A multi-stage build: the `maven:3.9.11-eclipse-temurin-17` stage copies `pom.xml` first and runs `dependency:go-offline`, so dependency downloads stay cached as a separate layer, then builds the JAR. The `eclipse-temurin:17-jre-jammy` runtime stage copies only the JAR, runs it as the non-root user `calculator` and declares `EXPOSE 8080`. A deny-by-default `.dockerignore` sends only `pom.xml` and `src/` to the build context.

Compose. Two services on the user-defined network `calculator`. *calculator* runs the image Jenkins passes in `CALCULATOR_IMAGE`, sets the `SERVER_PORT` and `JAVA_TOOL_OPTIONS` environment variables, and has a health check on `/actuator/health` and `restart: unless-stopped`. *proxy* (`nginx:1.28-alpine`) starts only after the API is healthy, has its own health check, and is the only service published (`127.0.0.1:8081`). Nginx resolves `calculator` through Docker DNS, so it keeps working when a deploy replaces the API container.

7. Challenges Faced

- **Java version mismatch:** `java -version` showed 17, but Maven used `JAVA_HOME` (Corretto 11) and failed with *release version 17 not supported*. Named `JDK17`/`Maven3` tools in the Jenkinsfile fixed this.
- **Port 8080 conflict:** another project already published `0.0.0.0:8080`, so `localhost:8080` did not reach Jenkins. I used `127.0.0.1:8080` for Jenkins and moved the proxy to 8081 (`APP_PORT`).
- **PATH for the Jenkins service:** a macOS service does not inherit the terminal's `PATH`, so the Jenkinsfile adds the Homebrew and Docker Desktop binary directories.
- **Proxy after redeploy:** a new API container gets a new IP, but a plain `proxy_pass` resolves the name only once. A variable upstream with `resolver 127.0.0.11` makes Nginx re-resolve it.
- **Gmail SMTP 535:** the first saved credential was rejected. A Gmail app password (TLS, port 587) fixed it.

8. Results

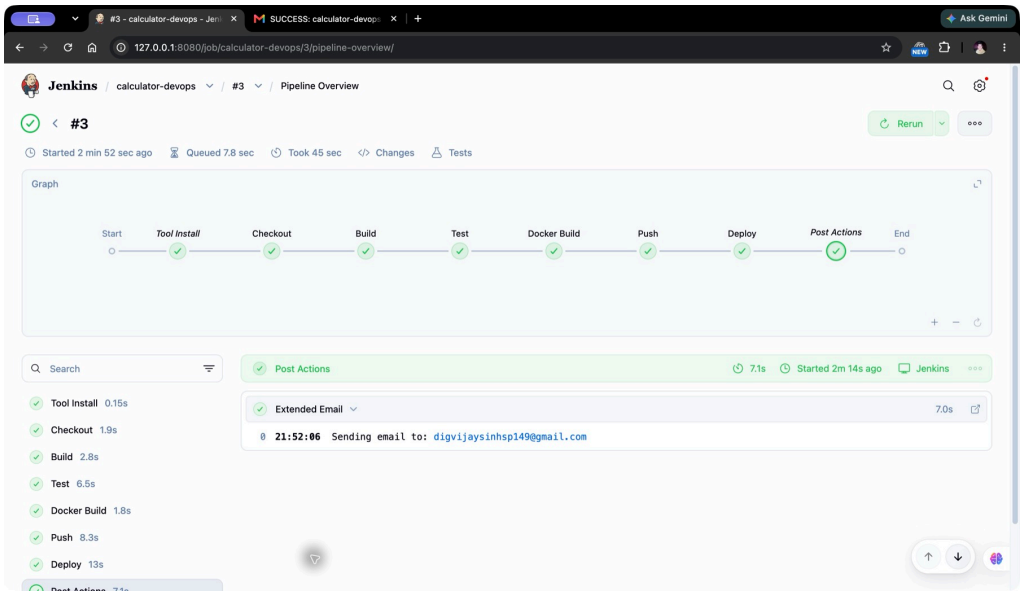
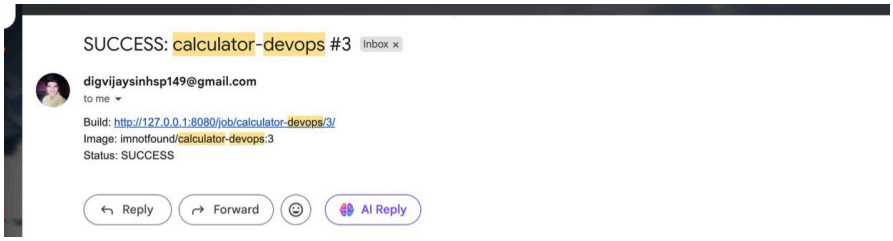
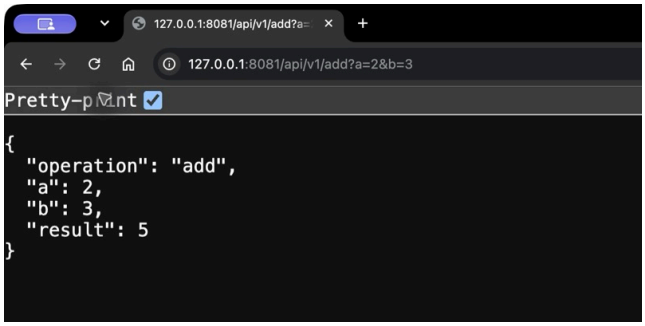


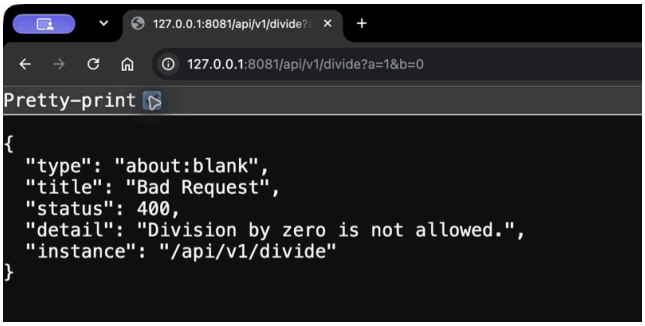
Figure 2: Jenkins build #3, started automatically by an SCM change: every stage passed



(a) Success email for build #3



(b) `GET /api/v1/add?a=2&b=3`



(c) `GET /api/v1/divide?a=1&b=0` gives 400

Figure 3: Email notification and API responses through the Nginx proxy

```
$ docker compose ps
NAME                            IMAGE                              STATUS                  PORTS
calculator-devops-calculator-1  imnotfound/calculator-devops:4  Up 7 minutes (healthy)  8080/tcp
calculator-devops-proxy-1       nginx:1.28-alpine                  Up 7 minutes (healthy)  127.0.0.1:8081->80/tcp
```

Listing 1: Running stack after build #4

9. Conclusion

A push to `main` is now built, tested, published as a versioned image and deployed as a health-checked two-container stack, and the result is emailed; build #3 ran this way with no manual steps. For the Capstone I plan to add a GitHub webhook, image rollback, Ansible, Kubernetes, and Prometheus/Grafana monitoring.